DESAFÍO 2: UDE@BOOK

Paulina Quintero Arenas, Luis Castillo Chicaiza

Facultad de Ingeniería Universidad de Antioquia
Informatica II
2026-02

1. ANÁLISIS DEL PROBLEMA Y CONSIDERACIONES PARA LA ALTERNATIVA DE SOLUCIÓN PROPUESTA

Para abordar el desarrollo de la capa lógica de este proyecto Ude@Book identificamos varios desafíos. La gestión eficiente de un volumen de información la cual está interconectada (Usuarios, amistades, álbumes, fotos y mensajes) con la limitación de no poder usar la librería STL de C++, esto nos llevo a crear desde cero nuestra propia clase *Vector¡T¿*para poder manejar arreglos dinámicos. Esta solución, nos permitió tener un control más preciso sobre el dimensionamiento de los arreglos para los diferentes datos manejados.
Pensando en el tema de la eficiencia tenemos las siguientes consideraciones, para el manejo de los amigos (contactos) decidimos implementar una matriz de adyacencia (*matrizAmistad*) la cual nos permite consultar si 2 personas son amigas sin tener que consultar persona por persona sino que podemos acceder a esto con una formula matemática directa (similar al desafío 1 para acceder a una f,c especifica) que nos indica una posición de la matriz donde con 0 o 1 representamos si 1: son amigos y 0: no son amigos. Con el uso de esta misma matriz, podemos calcular los amigos en común mediante un solo ciclo en los usuarios registrados comparando simultáneamente las conexiones entre ambas personas. Por estas razones, decidimos mantener la clase de usuario muy ligera, donde, este no guarda referencias sobre sus amigos sino que deja esto a la matriz para evitar redundancia en los datos.

Adicionalmente, optamos por no almacenar en memoria cadenas redundantes o que no se requieren para la ejecución normal del código, como por ejemplo, las rutas de las fotos no se almacenan sino que se construyen dinámicamente en tiempo de ejecución.

2. DIAGRAMA DE CLASES DE LA SOLUCIÓN PLANTEADA

Dado la alta cantidad de componentes que tiene este proyecto y las funciones de las clases, este queda bastante extenso, por esta razón, el diagrama se puede encontrar en el anexo 1 encontrado en el GitHub o en la figura 1 al final de este informe. La arquitectura del código se estructura alrededor de

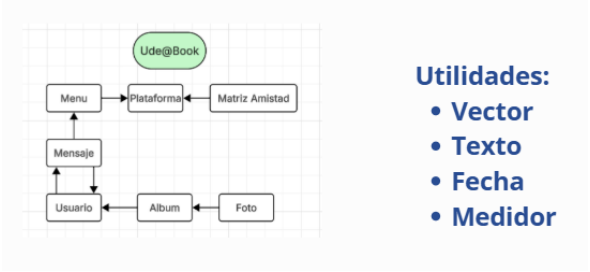


Fig. 1. Diagrama general de las clases usadas

11 clases principales y otras de utilería para el empaquetamiento y organización de la información. Las clases usadas son las siguientes:

- **Plataforma:** Esta es la unica clase que agrupa los conjuntos globales de datos y se encarga de centralizar toda la lógica del sistema (validar amistades, sugerencias, enviar mensajes). Esta, ofrece métodos para que la interfaz interactue con los datos sin necesidad de conocer como se almacenan en memoria.

- **Menu:** Gestiona la interacción del usuario por medio de la terminal. Captura entradas, las filtra y deja el tema de procesamiento a la clase *plataforma*. Esta clase, garantiza que la lógica del sistema no se mezcle con las instrucciones de entrada y salida.

- **Usuario:** Reúne toda la información personal de la persona. Para evitar referencias redundantes de memoria, un usuario no guarda punteros directos a sus amigos o mensajes, en vez de eso, administra la composición de los albumen y mantiene listas de indices numéricos que apuntan a sus mensajes (enviados y recibidos) y a sus solicitudes de amistad pendientes.

- **Álbum y Foto:** Álbum actúa como un contenedor que define la privacidad (publico, amigos y privado) del contenido de un usuario. Foto almacena datos como fecha de captura, tamaño en bytes y cantidad de likes). foto construye dinamicamente la ruta abdoluta de los archivos del sistema.

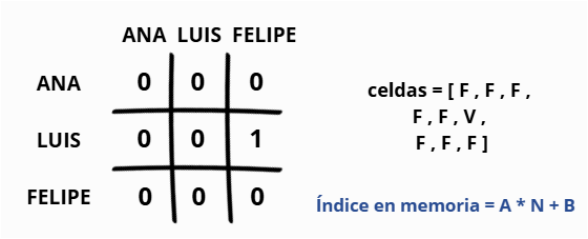


Fig. 2. MatrizAmistad y su estructura plana

- **Mensaje:** Esta clase modela la comunicación entre dos personas. Contiene el texto del mensaje, fecha y hora exacta de envío. Adicionalmente contiene un estado booleando de lectura. Cada objeto se asocia lógicamente a sus usuarios mediante identificadores del emisor y receptor.
- **MatrizAmistad:** Simula una matriz NxN (mediante un arreglo dinámico), esta clase contiene las operaciones necesarias para establecer conexiones mutuas, consultar amistades y calcular la intersección de amigos en común para recomendar nuevas amistades, la figura 2 muestra como se ve esta estructura booleana y la formula para acceder a los indices de ésta.
- **Persistencia:** Se encarga de transformar los objetos a registros de texto plano y viceversa. Esta permite realizar cargas masivas al iniciar el sistema y escrituras inmediatas (anexar líneas o sobrescribir archivos) cada vez que ocurre un cambio en la Plataforma.
- **Vector:** La mas importante, ya que es la plantilla fundamental del proyecto que reemplaza completamente STL. esta clase, gestiona los arreglos dinámicos gestionando el redimensionamiento automático, garantizando copias de memoria y destruyendo los bloques que no se vayan a usar más, de forma segura.
- **Fecha:** Estandariza validaciones temporales, la captura del reloj del sistema y el empaquetado de cadenas.
- **Texto:** Asegura el recorte de espacios en blanco y el descarte de caracteres corruptos como el de salto de linea ($\backslash r$)
- **Medidor:** Se encarga de el consumo de bytes en estructuras dinámicas y la acumulación de iteraciones.

3. ALGORITMOS ESENCIALES

Como se mencionó en la sección anterior, debido a la restricciones de diseño para esta practica, nos vimos en la necesidad de implementar nuestras lógicas propias para algunos procesos, a continuación se hace una descripción de ellos:

- **Sugerencias de amistad:** Para esto, es necesario identificar los mejores candidatos teniendo en cuenta la mayor cantidad de amigos en común. Para esto, creamos en *Plataforma::sugerencias* arreglos dinámicos de los mejores candidatos, si el usuario evaluado tiene más amigos en común que el peor candidato actual, lo insertamos empujando a los demás hacia abajo. El criterio de desempate evalúa primero los amigos en común y luego el orden alfabético estricto.
- **Selección de fotos mediante ruleta ponderada:** Al iniciar sesión en la plataforma, se debe mostrar 3 fotos aleatorias de amigos pero dándole más prioridad a los que tienen más amigos en común. En *Plataforma::girarRuleta* implementamos un algoritmo de ruleta"basado en probabilidad acumulada. Primero, el algoritmo recorre a los amigos elegibles sumando sus pesos asignados (2, 5 o 7 según las coincidencias). Luego, genera un número aleatorio uniforme entre 0 y el peso total acumulado. Finalmente, vuelve a recorrer la lista acumulando los pesos uno por uno; en el momento en que esta suma parcial supera el número aleatorio, selecciona a ese amigo.
- **Búsqueda binaria por rango de fechas:** Para buscar mensajes recibidos en un rango de tiempo, evitamos la búsqueda exhaustiva, ya que esto incrementaría la métrica de iteraciones del sistema. Dado que el módulo de *Persistencia* hace que los mensajes se carguen en orden cronológico, implementamos un algoritmo de búsqueda binaria en *Plataforma::buscarRango*. El algoritmo divide el conjunto a la mitad hasta encontrar el primer mensaje cuya fecha es mayor o igual a la fecha de inicio ingresada. Una vez encontrado ese punto de partida, itera hacia adelante agregando mensajes hasta que la fecha supera la de fin.
- **Gestión de capacidad dinámica:** Al no poder usar std::vector existe un problema en la gestión de arreglos que crecen con el tiempo. Como se menciona anteriormente, creamos nuestro propia clase con el método *agregar* nos permite ir aumentando el tamaño de la memoria dinámica en un rango seguro. Cuando el número de elementos (tam) alcanza la capacidad reservada (cap), el algoritmo crea un nuevo bloque de memoria con el doble de capacidad (cap * 2), copia los elementos existentes, agrega el nuevo valor y finalmente destruye de forma segura (delete[]) el bloque de memoria viejo. Esto evita tener que redimensionar el arreglo por cada elemento ingresado.

4. PROBLEMAS DE DESARROLLO QUE AFRONTADOS Y EVOLUCIÓN DE LA SOLUCIÓN

En primer lugar el problema que más resalta es la restricción en el uso de STL y el agregarle control de rendimiento y mantenerlo bajo, a continuación se describen brevemente estos y sus soluciones implementadas:

- Al inicio de la implementación experimentamos fugas de memoria y errores del programa al pasar los objetos *Vector* y *matrizAmistad* a diferentes funciones o al reasignar arreglos dinámico, esto debido a que el compilador generaba copias que apuntaban a la misma dirección de memoria, entonces al momento de liberar la memoria quedaban esas copias como punteros colgantes. Para solucionar esto definimos constructores para realizar las copias, sobrecargamos el operador de asignación (operador=) y también creamos destructores seguros para todas nuestras clases que hacen uso de el Heap.

- Al probar la lectura de los archivos txt notamos que las validaciones de las cadenas y conversiones fallaban. Identificamos que los archivos generados añadían el carácter de retorno $\backslash r$ invisible al final de cada línea. Para resolverlo, creamos un método *Texto::leerLinea*, el cual detetca y elimina este caracter antes de procesar la cadena.

- El hecho de cuantificar la memoria e iteraciones sin agregar operaciones que nos incrementaran más estas métricas fue un reto ya que se debía pasar el objeto medidor entre funciones. Para esto creamos la clase *medidor* puramente estática, donde no solo calculamos la memoria dinámica (variables locales,llamadas a aleatorios) también necesitamos usar el operador *sizeof()* sobre cada variable local y apuntador para reportar su cantidad de bytes a *medidor*.

- Para la búsqueda binaria, fue necesario que todos los mensajes estuvieran en orden cronológico. Tuvimos el problema de que, si el reloj del SO estaba desincronizado o se enviaban varios mensajes .en el mismo instante"la nueva fecha capturada por *Fecha::ahora()* podía ser inferior o igual a la del último mensaje cargado ocasionando daños en el orden cronológico. Lo solucionamos añadiendo un atributo *reloj* en la *Plataforma* que actúa como un çaché temporal"; si la hora del sistema es anterior a la de nuestro último mensaje, forzamos a que el nuevo mensaje herede la fecha del reloj interno, salvaguardando así la integridad de la estructura para futuras búsquedas.

4.1. Link del repositorio

`https://github.com/LuisC645/ude-book.git`

4.2. Link del Video

`https://youtu.be/INSWlQffuJs`

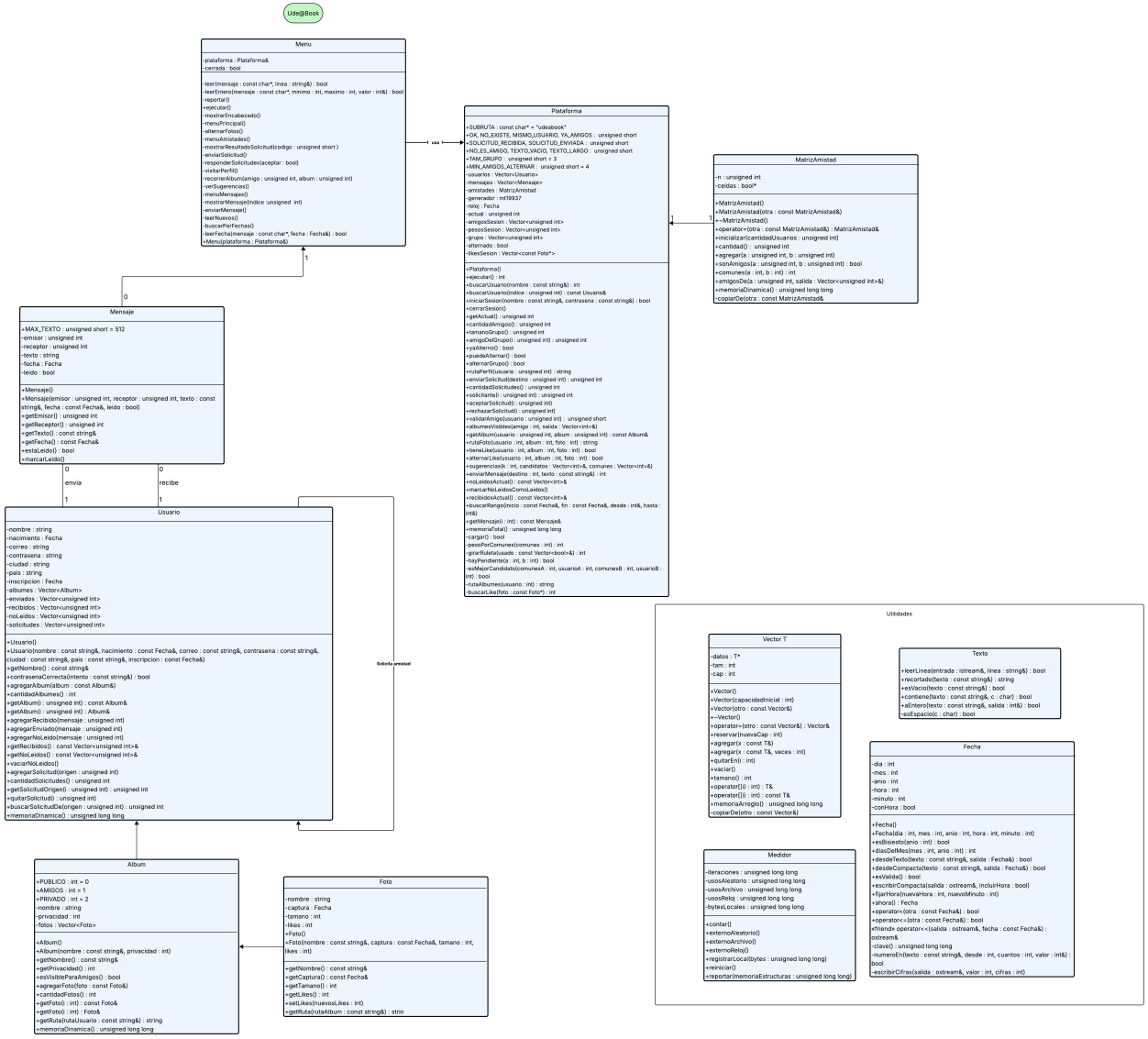


Fig. 3. Diagrama de clases del sistema UdeBook.